

# SYSTEM ANALYSIS DOCUMENTATION (SAD)

## UMHackathon 2026

[umhackathon@um.edu.my](mailto:umhackathon@um.edu.my)

---

### SYSTEM ANALYSIS DOCUMENTATION (SAD)

**Project Name:** BizMind – AI Business Advisor

**Domain:** AI for Economic Empowerment & Decision Intelligence

#### 1. Introduction

This section introduces the technical solution for **BizMind**, as highlighted in the Product Requirement Documentation.

##### 1.1 Purpose

The system analysis document highlights the technical scope and design-related decisions behind the BizMind development. The key elements covered are:

1. **Architecture:** A Client-Server model using FastAPI and a specialized LLM Service Layer.
2. **Data Flows:** How raw sales data moves from the UI, through mathematical processing in the backend, to the Z AI GLM, and back to the user.
3. **Model Process:** The workflow from data ingestion to partnership matching and strategic advice generation.
4. **Role of Reference:** This serves as a guide for developers to understand how the Python backend and Z AI API interact.

##### 1.2 Background

**Previous Version:** Small business owners previously used manual ledger books or basic Excel sheets. These methods are "static"—they show the numbers but do not explain the "why" or suggest "what's next," requiring a human expert to interpret the data.

**Changes in Major Architectural Components:** BizMind introduces an **Automated Logic Layer**. Instead of a user just looking at a spreadsheet, the backend now performs real-time variance analysis (Weekday vs. Weekend) and injects these facts into a High-Reasoning LLM (**Z AI GLM**).

**New Capabilities:** \* **Contextual Reasoning:** The system explains why sales changed based on the business type.

- **Automated Partnership Matching:** A JSON-based database search that connects users to local service providers.

---

## 2. System Architecture & Design

### 2.1 High-Level Architecture

BizMind follows a **Modular Monolith** architecture using Python's FastAPI. It is designed to be lightweight yet capable of complex reasoning.

- **Frontend Layer:** Built with HTML5/JS, focusing on a "Terminal" experience for decision-making.
- **Service Layer (The Backend):** This is where the `compute_sales_metrics` logic resides. It ensures that the AI receives "clean" mathematical facts.
- **Intelligence Layer: Z AI GLM (*ilmu-glm-5.1*)** acts as the reasoning engine for generating advice and explaining partnership benefits.

### 2.2 Data Flows

The data flow follows a **Linear Intelligence Pipeline**:

1. **Input:** User enters description, `weekday_sales`, and `weekend_sales`.
2. **Transformation:** Backend calculates `total_sales`, `avg_daily`, and `pct_change`.
3. **Augmentation:** These metrics are added to the prompt (Prompt Engineering).
4. **Reasoning:** Z AI GLM processes the prompt and generates a structured advice report.
5. **Output:** The JS Frontend renders the AI response and partnership matches.

---

## 3. Functional Requirements & Scope

### 3.1 Requirement Traceability

| ID    | Requirement          | Status    | Technical Implementation                                      |
|-------|----------------------|-----------|---------------------------------------------------------------|
| FR-01 | Sales Analysis       | Completed | <code>compute_sales_metrics</code> function in Python.        |
| FR-02 | AI Advice            | Completed | Integration with <i>ilmu-glm-5.1</i> via <code>httpx</code> . |
| FR-03 | Partner Matching     | Completed | JSON keyword-matching algorithm.                              |
| FR-04 | Decision Explanation | Completed | Prompt engineering requiring AI to explain the "Why."         |

### 3.2 Non-Functional Requirements (NFRs)

- **Reliability:** The backend validates all numerical inputs before calling the AI to prevent crashes.
- **Performance:** AI inference is limited to 800 tokens to ensure response times under 3 seconds.
- **Security:** CORS middleware is configured to allow secure communication between the frontend and backend.

---

## 4. Monitor, Evaluation & Dependencies

### 4.1 Assumptions & Dependencies

© UMHackathon 2026

Email: [umhackathon@um.edu.my](mailto:umhackathon@um.edu.my)

- **Assumption:** Users provide honest and accurate sales figures.
- **Z AI GLM API:** The system depends on this for reasoning. If the API is down, the system triggers a **Graceful Error State** showing only the math metrics.
- **Local Database:** `businesses.json` must be present in the root directory for matching to function.

#### 4.2 External Tools

| Tools                | Purpose                                    | Risks                                            |
|----------------------|--------------------------------------------|--------------------------------------------------|
| <b>Z AI GLM API</b>  | Reasoning and Natural Language Generation. | API Rate limits.                                 |
| <b>FastAPI</b>       | High-performance API hosting.              | Port 8000 conflicts.                             |
| <b>JSON Database</b> | Storage of partnership profiles.           | Limited scalability for thousands of businesses. |

#### 5. Project Management (Team Contribution)

| Name               | Role                       | Responsibilities                                                         |
|--------------------|----------------------------|--------------------------------------------------------------------------|
| <b>Mosa</b>        | Backend and AI integration | Building the backend as well as connecting the AI with the whole program |
| <b>Abdulrahman</b> | UI and documentations      | Frontend development and documents management                            |

#### 6. Time management

| Phase             | Days | Focus                   | Milestones                                                        |
|-------------------|------|-------------------------|-------------------------------------------------------------------|
| Research          | 1-2  | Validating the problem. | Finalising the concept and the decision intelligence workflow.    |
| Core development  | 3-7  | Parallel building       | Building the backend and the frontend.                            |
| <b>AI syncing</b> | 8-9  | Integrating Z AI        | Linked backend to <b>ilmu-glm-5.1</b> .                           |
| <b>Finalising</b> | 9-10 | Finishing the details   | Making the pitch and finishing the documentations for submission. |